PowerShell App Deploy Toolkit (PSADT) & MSI Tables

Introduction to PSADT

- Definition: Open-source framework written in PowerShell for application deployment
- Purpose: Standardize and simplify software installations/uninstallations
- Benefits: Saves time, ensures consistent process, integrates with Intune/SCCM/MDT
- Real-world usage: Deploying apps to hundreds of devices with minimal errors

Key Features of PSADT

- Pre-Built Functions: Execute-Process, Show-InstallationWelcome
- User Interaction: Prompts, warnings, progress bars
- Error Handling: Automatic logging to log files
- Custom Logic: Detect OS, user role, or application status
- Support for Silent Installs: Fully automated installs
- Rollback Support: Undo changes if install fails

PSADT Folder Structure

- Deploy-Application.ps1: Main script for customizing deployment
- AppDeployToolkitMain.ps1: Core functions & logic
- AppDeployToolkitExtensions.ps1: Custom add-on functions
- Files Folder: Main installation files
- SupportFiles Folder: Extra resources like configs/icons
- Logs Folder: Install logs for troubleshooting

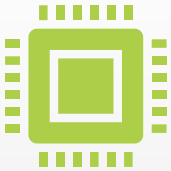
Best Practice: Keep folder names short and clear

Common PSADT Commands

Command	Explanation
Show-InstallationPrompt	Displays a custom message box to the user.
Show-InstallationWelcome	Shows a prompt to close applications before install.
Install-MSI	Installs an MSI file with specified parameters.
Uninstall-MSI	Uninstalls an MSI application by product code or name.
Execute-Process	Runs

Introduction to MSI

MSI = Microsoft Installer database format



Advantages: Easy repair/uninstall, version control, GPO deployments

Why MSI matters: Reliable structure, silent install with /quiet switch

Tools to work with MSI: Orca, InstEd, SuperOrca

Introduction to MSI Table

The **MSI Table** (short form Microsoft Installer Table) is a key data structure used primarily in computer systems, particularly within the **Advanced Configuration and Power Interface (ACPI)** and **Peripheral Component Interconnect** (**PCI**) environments.

Why Microsoft Installers Matter

1.Built-In Repair & Rollback

- If something breaks, MSI-based applications can **repair themselves**.
- During installation, a snapshot is taken; if it fails, the system can **rollback to its previous state**.
- This prevents partial installations or corrupted software states.

2. Efficient Uninstall and Version Control

- MSI keeps a complete record of installed components and registry entries.
- Supports **clean uninstalls**, **version upgrades**, and **patching**.
- Prevents "registry rot" and file leftovers that are common with manual setups.

3. Custom Actions and Scripting

- Developers can include **custom scripts, logic, or conditions** during installation—such as:
- License checks
- System requirement validation
- Pre- or post-installation scripts

4. Real-World Example

- Imagine a company deploying a custom CRM app to 500 employee PCs. With MSI:
- IT can **push the app** via Group Policy
- Users don't need to interact with the installer
- Any corrupted installation can be **self-repaired**
- Uninstalls are clean and reversible

Advantages of Microsoft Installers (MSI)

- **Rollback Support**

If installation fails, Windows Installer can **roll back** changes, leaving the system in a clean state.

Prevents half-installed or broken software scenarios.

- **Clean Uninstall**

MSI tracks every file, registry key, and setting added during installation.

Uninstalls cleanly and automatically removes all components, reducing system clutter.

- **Security**

MSI packages can be **digitally signed**, ensuring they are from a trusted source.

Windows Installer runs with elevated privileges but can be managed via policies to prevent abuse.

Reduces risk of malicious or corrupted installations.

- **Automatic Repair (Self-Healing)**

If an application file is deleted or corrupted, Windows Installer can **detect the issue and repair it automatically**.

Uses internal checks and installation logs to restore missing or damaged components.

Tools for Working with Microsoft Installers (MSI)

1. **Orca (Microsoft Tool)**

   What it is: A table editor for MSI databases.

   Who it's for: Advanced users and developers for debugging, editing, or viewing MSI internals.

   Key Features:

   - Edit MSI tables directly
   - View properties, custom actions, sequences, etc.
   - Great for reverse-engineering or troubleshooting installers

2. **InstallShield**

   What it is: An enterprise-grade commercial tool for creating MSI and other installer formats.

   Who it's for: Enterprise developers needing advanced features.

   Key Features:

   - Complex UI and scripting support
   - Multilingual installers
   - Code-signing and virtualization support

3. **MSIX Packaging Tool**

   What it is: A Microsoft tool to convert MSI, EXE, or other installers to **MSIX**, the modern packaging for

   Who it's for: Developers migrating to MSIX.

   Key Features:

   - Converts MSI → MSIX
   - GUI-based wizard
   - Available in Microsoft Store

Common MSI Tables & Purpose

- **File Table** – List of files to install and where to put them.
- **Directory Table** – Folders to create during installation.
- **Feature Table** – Groups of software parts that can be installed.
- **Component Table** – Keeps related files and settings together.
- **Registry Table** – Adds or changes Windows Registry settings.
- **Shortcut Table** – Creates shortcuts on Desktop or Start Menu.
- **CustomAction Table** – Runs extra commands or scripts during install.
- **Upgrade Table** – Removes old versions before installing new ones.
- **LaunchCondition Table** – Checks system requirements before starting install.
- **InstallUISequence Table** – Controls the order of setup wizard steps.

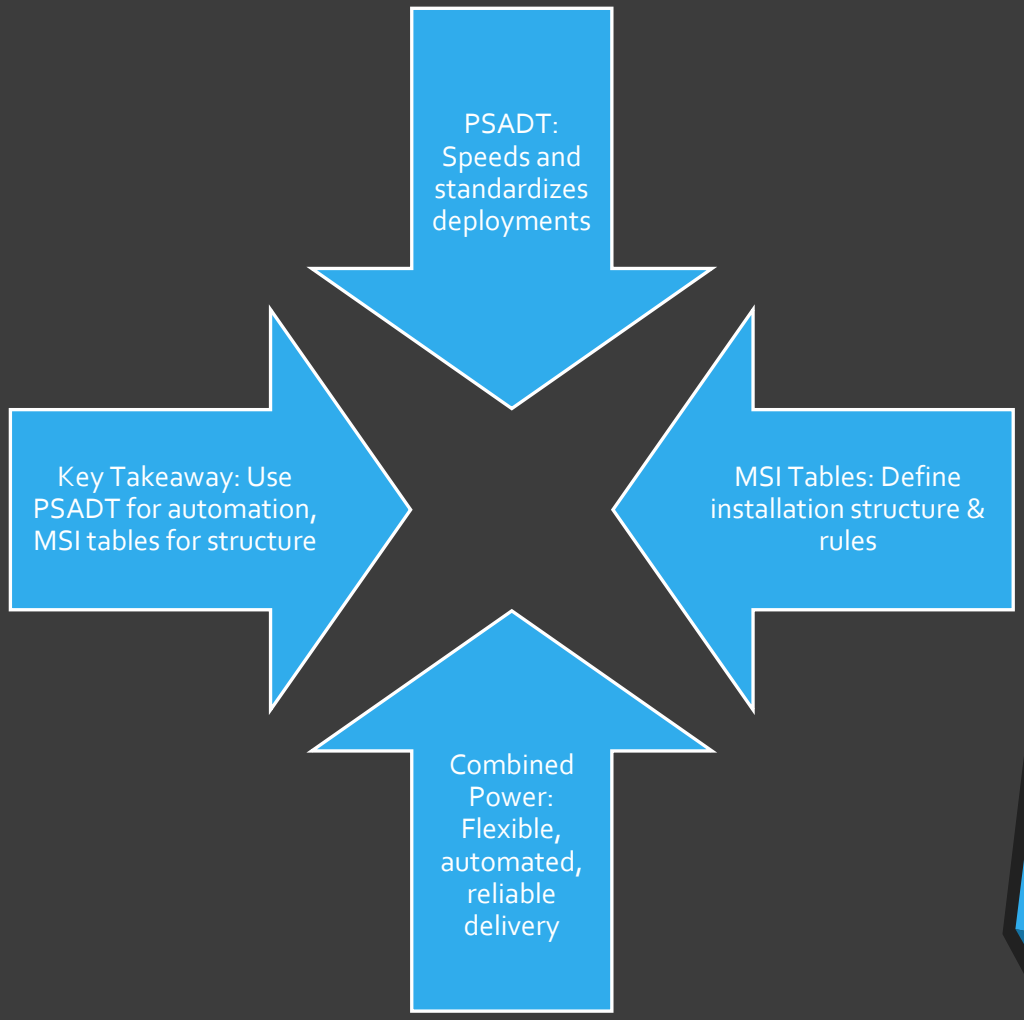
Why They Matter in Packaging

- You can **customize an MSI** by modifying tables or adding a transform file (`.MST`).
- Helps in **troubleshooting** install failures.
- Let you **find the Product Code** for detection methods.
- Allows you to **remove dialogues, add registry keys, and change properties**

Why Combine PSADT with MSI Tables

- PSADT wraps MSI installs with pre/post tasks
- User-friendly messages instead of silent background installs
- Example Workflow:
- 1. Check if old version installed
- 2. Uninstall using MSI table info
- 3. Install new MSI package
- 4. Log results and notify user
- Benefits: Flexibility, works with MSI & EXE, reduces failures

Summary


PSADT: Speeds and standardizes deployments
Key Takeaway: Use PSADT for automation, MSI tables for structure
MSI Tables: Define installation structure & rules
Combined Power: Flexible, automated, reliable delivery